

IvanChain

Candidate name: Ivan Leung

Candidate number: 4022

Centre name: 22245

Qualification code: H446

Year: 2026

Contents

1. Analysis	3
1.1. The Problem	4
1.2. Research	5
1.3. Initial Features	8
1.4. Intital Sucessful Criteria	9
1.5. Stakeholders	10
1.5.1. Survey	10
1.5.2. Interview	10
1.6. Features	11
1.7. Extensions	12
2. Iterations	13
2.1. Iteration 1	14
2.1.1. Design	14
2.2. Decomposition	15
2.2.1. Testing	15
2.2.2. Evaluation	15
2.3. Iteration 2	16
2.3.1. Testing	16
2.3.2. Evaluation	16
2.4. Iteration 3	17
2.4.1. Testing	17
2.4.2. Evaluation	17
2.5. Iteration 4	18
2.5.1. Testing	18
2.5.2. Evaluation	18
2.6. Iteration 5	19
2.6.1. Testing	19
2.6.2. Evaluation	19
2.7. Algorithms	20
2.8. Test Data	21
2.9. Data Validation	22
3. Evaluation	23

1. Analysis

1.1. The Problem

Despite many people are investing in cryptocurrency markets, most of them do not understand how blockchain technology (the technology which powers cryptocurrencies) works, and there is a lack of educational resources available to help them learn. The goal is to create a platform where users can learn about blockchain technology, how it works, and how to use it.

According to [fenews.co.uk](https://www.fenews.co.uk)^[1],

¹<https://www.fenews.co.uk/skills/cryptocurrency-should-part-of-the-national-curriculum/>

1.2. Research

Blockchain is a distributed ledger technology that allows secure and transparent transactions without the need for a central authority. This creates a secure, members-only network, ensuring accurate and timely data access. It is often used in technologies such as cryptocurrencies, supply chain management, and Non-Fungible Tokens (NFT). Blockchains are decentralized, meaning that they are not controlled by a single entity, and they are immutable, meaning that once data is added to the blockchain, it cannot be changed or deleted. This also means it can only be stored in a peer-to-peer network, which is a network of computers that are connected to each other and can communicate with each other, and they all have equal status.

Mining is the process of adding transactions to the blockchain, and it is done by solving complex mathematical problems. This process is known as Proof of Work (PoW), and it is used to secure the blockchain and prevent double spending. In PoW, miners compete to solve a mathematical problem, and the first miner to solve the problem gets to add the next block to the blockchain and is rewarded with a certain amount of cryptocurrency. This process is resource-intensive and requires a lot of computational power, which is why it is often done by specialized hardware known as ASICs (Application-Specific Integrated Circuits). This algorithm is used in many cryptocurrencies, such as Bitcoin and Litecoin, to secure the blockchain and prevent double spending. Proof of Stake (PoS) is an alternative to PoW, where miners are chosen to add the next block to the blockchain based on the amount of cryptocurrency they hold. This means that the more cryptocurrency a miner holds, the more likely they are to be chosen to add the next block to the blockchain. This process is less resource-intensive and does not require specialized hardware, making it more accessible to a wider range of users. This algorithm is used in many newer cryptocurrencies, such as Ethereum 2.0, which is transitioning from PoW to PoS.

Research in Blockchain Technology:

This video by 3Blue1Brown explains how blockchain technologies work behind the scene and different algorithms to power it

I found this blockchain simulator CLI on GitHub (<https://github.com/0xs34n/blockchain>). I have downloaded

```
ivanextrastuff@Acer-Aspire:~/code/b_simulator/blockchain$ npm start

> blockchain-cli@2.0.0 start /home/ivanextrastuff/code/b_simulator/blockchain
> node ./index.js

Welcome to Blockchain CLI!

Commands:

  help [command...]  Provides help for a given command.
  exit               Exits application.

blockchain → help

Commands:

  help [command...]  Provides help for a given command.
  exit               Exits application.
  connect <host> <port> Connect to a new peer. Eg: connect localhost 2727
  discover            Discover new peers from your connected peers.
  blockchain          See the current state of the blockchain.
  peers              Get the list of connected peers.
  mine <data>         Mine a new block. Eg: mine hello!
  open <port>         Open port to accept incoming connections. Eg: open 2727

blockchain → mine IvanBlock1
blockchain → mine IvanBlock2
blockchain → blockchain
Blockchain {
  blockchain:
    [ Block {
      index: 0,
      previousHash: '0',
      timestamp: 1508270000000,
      data: 'Welcome to Blockchain Demo 2.0!',
      hash: '000dc75a315c77a1f9c98fb6247d03dd18ac52632d7dc6a9920261d8109b37cf',
      nonce: 604 },
      Block {
        index: 1,
        previousHash: '000dc75a315c77a1f9c98fb6247d03dd18ac52632d7dc6a9920261d8109b37cf',
        timestamp: 1757326219903,
        data: 'IvanBlock1',
        hash: '000487dbd92876c509ca612969413c9398cd7a0a936ef1a716ef3d634c3d93c5',
        nonce: 5482 },
      Block {
        index: 2,
        previousHash: '000487dbd92876c509ca612969413c9398cd7a0a936ef1a716ef3d634c3d93c5',
        timestamp: 1757326223309,
        data: 'IvanBlock2',
        hash: '000c5f9316b3c17b07ce6f9fe440ef29036529d0268a55af9457e8a43e413b09',
        nonce: 8579 } ],
  difficulty: 3 }
blockchain →
```

```

ivanextrastuff@Acer-Aspire:~/code/b_simulator/blockchain$ npm run start
> blockchain-cli@2.0.0 start
> node ./index.js

Welcome to Blockchain CLI!

Commands:

  help [command...]  Provides help for a given command.
  exit               Exits application.

blockchain → help

Commands:

  help [command...]  Provides help for a given command.
  exit               Exits application.
  connect <host> <port> Connect to a new peer. Eg: connect localhost 2727
  discover            Discover new peers from your connected peers.
  blockchain          See the current state of the blockchain.
  peers              Get the list of connected peers.
  mine <data>         Mine a new block. Eg: mine hello!
  open <port>         Open port to accept incoming connections. Eg:
                     open 2727

blockchain → open 2024
Listening to peers on 2024
blockchain → peers
null

blockchain → discover
blockchain →

/home/ivanextrastuff/code/b_simulator/blockchain/P2p.js:39
  throw err;
  ^
Error: Peer did not return any candidates
    at /home/ivanextrastuff/code/b_simulator/blockchain/node_modules/peer-exchange/lib/swarm.js:258:21
    at PXP.<anonymous> (/home/ivanextrastuff/code/b_simulator/blockchain/node_modules/pxp/lib/peer.js:439:9)
    at Object.onceWrapper (events.js:421:26)
    at PXP.emit (events.js:314:20)
    at PXP.onMessage (/home/ivanextrastuff/code/b_simulator/blockchain/node_modules/pxp/lib/pxp.js:66:21)
    at Pumpify.emit (events.js:314:20)
    at addChunk (/home/ivanextrastuff/code/b_simulator/blockchain/node_modules/readable-stream/lib/_stream_readable.js:201:12)
    at readableAddChunk (/home/ivanextrastuff/code/b_simulator/blockchain/node_modules/readable-stream/lib/_stream_readable.js:278:11)
    at Pumpify.Readable.push (/home/ivanextrastuff/code/b_simulator/blockchain/node_modules/readable-stream/lib/_stream_readable.js:245:10)
    at Pumpify.Duplexify._forward (/home/ivanextrastuff/code/b_simulator/blockchain/node_modules/duplexify/index.js:170:26)
ivanextrastuff@Acer-Aspire:~/code/b_simulator/blockchain$

ivanextrastuff@Acer-Aspire:~/code/b_simulator/blockchain$ npm run start
> blockchain-cli@2.0.0 start
> node ./index.js

Welcome to Blockchain CLI!

Commands:

  help [command...]  Provides help for a given command.
  exit               Exits application.

blockchain → mine IvanBlock
blockchain → blockchain
Blockchain {
  blockchain: [
    Block {
      index: 0,
      previousHash: '0',
      timestamp: 1508270000000,
      data: 'Welcome to Blockchain Demo 2.0!',
      hash: '000dc75a315c77a1f9c98fb6247d03dd18ac52632d7dc6a9920261d8109b37cf',
      nonce: 604
    },
    Block {
      index: 1,
      previousHash: '000dc75a315c77a1f9c98fb6247d03dd18ac52632d7dc6a9920261d8109b37cf',
      timestamp: 1758034508869,
      data: 'IvanBlock',
      hash: '00053d4d9787be5aff2d27dc025ffae15dde1f769a4bba00d7ca64c11b25f93',
      nonce: 1835
    }
  ],
  difficulty: 3
}
blockchain → connect localhost 2024
blockchain → peers
localhost
blockchain → []

```

In this image, Sean's Blockchain simulator has shown the connection between different ports in local host. However, there is not any features that allow different ports to interact, like trading blocks. It also doesn't allow user to see other's user blocks.

1.3. Initial Features

The solution will be developed in an agile way. This means that it will be developed in different iterations. The analysis of requirements has been done in this section (Section 1). In each iterations I will

- Design the prototype for that iteration
- Build the prototype
- Asking for User Feedback
- Refine prototype

Each feature of the solution might be improved throughout the iterations

1.4. Intital Sucessful Criteria

These are the initial Success Criteria (SC) which is what I am aiming for while developing a MVP (Minimum Viable Product). As I am developing in an agile methodology, the Success Critea might be ammended or new Success Criteria might be added in the process of iterations.

SC	Target	Parameters
$\pi h \frac{D^2 - d^2}{4}$	h : height D : outer radius d : inner radius	$\frac{\sqrt{2}}{12}a^3$
a : edge length		

1.5. Stakeholders

People who will be using the system, including developers, users, and administrators. This also includes the people who would like to invest in cryptocurrencies, allowing them to learn about the technology and how it works. By understanding the underlying principles of blockchain

1.5.1. Survey

1.5.2. Interview

1.6. Features

The platform will have the following features:

- Introduction to the platform, allowing users to learn about blockchain technology and how it works.
- A graph representation of the users and their relationships, allowing users to see how they are connected to each other.
- A visualisation of the blockchain, allowing users to see how the blockchains are connected to each other.
- Allowing users to create a new block, add a transaction, and verify a transaction.
- Allowing users to transact with each other, to move the blocks around between users.
- Checking the blocks in a chain
- Checking the users' profiles
- Adding users to the system
- Settings page allowing users to modify the UI for better aesthetics, such as dark mode
- Allow removing users from the system
- Allowing users to view the transactions between users
- Allowing users to make transactions between users.

1.7. Extensions

The platform can be extended to include more features, such as:

- A visualisation of how blocks are created by Proof of Work (PoW), allowing users to see how blocks are added to the blockchain.
- Implementing the Proof of Stake (PoS) algorithm, allowing users to see how blocks are added to the blockchain using PoS.
- A tutorial system that allows users to learn about controlling the blockchain, such as how to create a new block, how to add a transaction, and how to verify a transaction.
- A visualisation of what happens when a blockchain is forked, allowing users to see how the blockchain splits into two separate chains, and how this can be resolved.
- Showing different users are mining at the same time and competing to add the next block to the blockchain.
- Visualisation on how public and private key pairs play a role in the transaction process, including how they are used to sign transactions and verify ownership.
- Introducing multiple chains and visualising the conflict

2. Iterations

2.1. Iteration 1

2.1.1. Design

The figure below shows the user interface design of different sections of the platform, including the Introduction Page, Main Page, Users Page, Chains Page, Mining Page, Transactions Page, and the Settings Page.

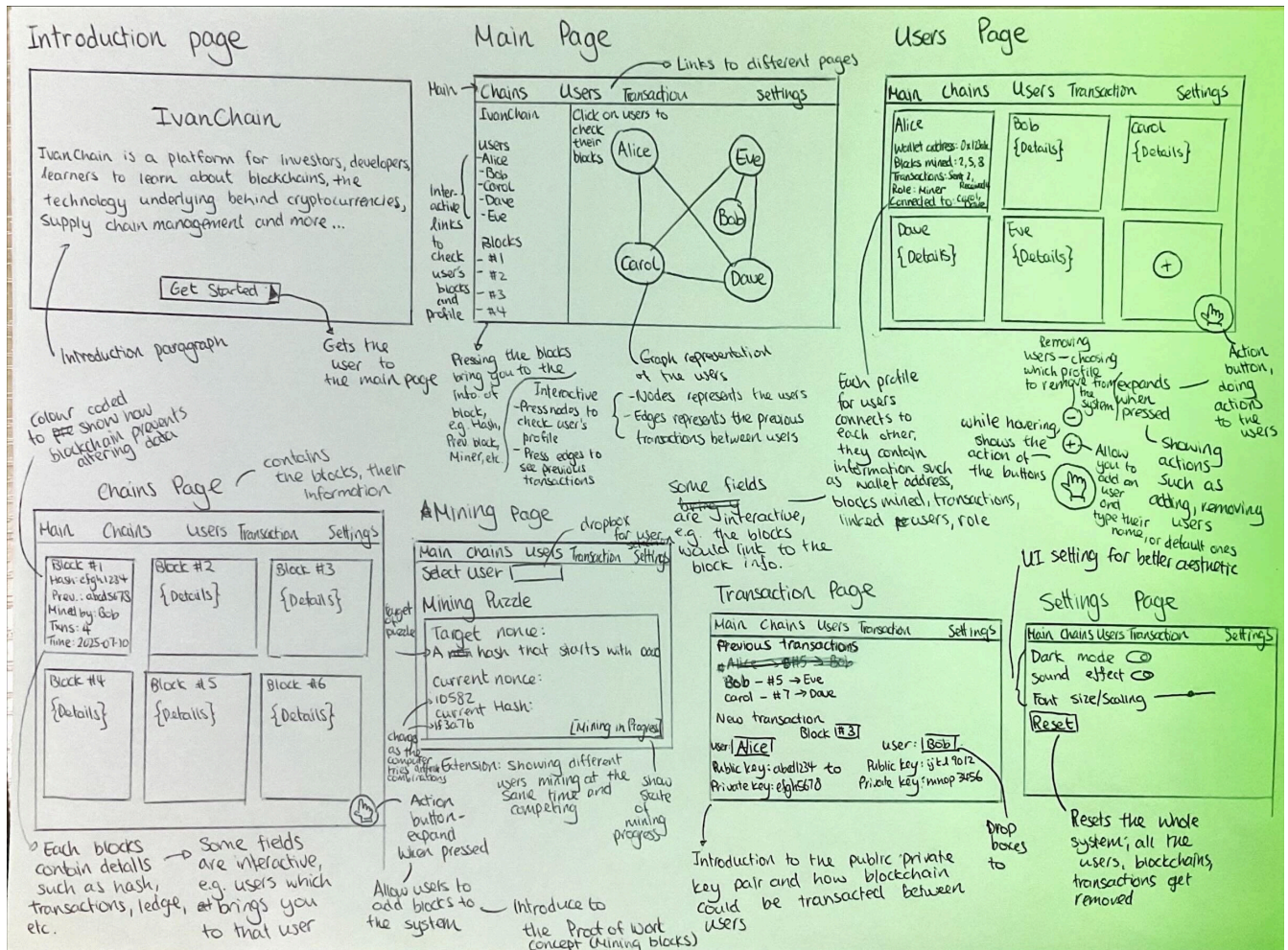


Figure 1: User interface of the blockchain simulator.

2.2. Decomposition

TODO:Justify for decomposition

2.2.1. Testing

2.2.2. Evaluation

2.3. Iteration 2

2.3.1. Testing

2.3.2. Evaluation

2.4. Iteration 3

2.4.1. Testing

2.4.2. Evaluation

2.5. Iteration 4

2.5.1. Testing

2.5.2. Evaluation

2.6. Iteration 5

2.6.1. Testing

2.6.2. Evaluation

2.7. Algorithms

2.8. Test Data

2.9. Data Validation

3. Evaluation